Соединение таблиц (SQL Joins и операции над множествами)

Шаблон запроса

```
SELECT
    column_list | *                    -- какие столбцы выбираем
FROM
    table_name                         -- из какой таблицы
[WHERE condition]                      -- фильтрация строк
[GROUP BY grouping_column]             -- группировка
[HAVING group_condition]               -- фильтрация групп
[ORDER BY column [ASC|DESC]]           -- сортировка
[LIMIT n]                              -- ограничение количества строк
[OFFSET m];                            -- смещение (например, для пагинации)
```

Порядок выполнения запроса

FROM + JOIN

Сначала выбираются таблицы и соединяются между собой. Именно здесь формируется полный набор строк для дальнейшей обработки.

WHERE

Применяются условия фильтрации строк (до группировок и агрегатов).

GROUP BY

Строки объединяются в группы по указанным колонкам.

HAVING

Фильтрация уже сгруппированных данных (например, убрать группы, где COUNT(*) < 5).

SELECT

Формируется список выходных столбцов, применяются вычисления, агрегатные функции (SUM, AVG, и т.д.).

DISTINCT

Удаляются дубликаты из результирующего набора.

ORDER BY

Сортировка итоговых строк.

LIMIT / OFFSET

Применяется ограничение и смещение для постраничного вывода.

Введение

Цель занятия:

- Освоить все основные способы соединения таблиц в SQL.
- Понять разницу между явными и неявными соединениями.
- Научиться использовать объединения, пересечения и разности множеств.

Ключевые вопросы:

- JOIN vs подзапросы
- Когда использовать INNER, OUTER, CROSS JOIN
- Как применять операции над множествами: UNION, EXCEPT, INTERSECT

Подзапросы

Подзапросы (Subqueries)

SELECT-запрос внутри другого запроса

Используется для фильтрации, вычисления агрегатов или как временная таблица

Может возвращать одно или несколько значений

Виды подзапросов

Скалярные — возвращают одно значение

Многострочные — возвращают набор строк

Коррелированные — зависят от внешнего запроса

Скалярный подзапрос

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Внутренний SELECT вычисляет среднюю зарплату всех сотрудников.
Внешний SELECT выбирает только тех сотрудников, у которых зарплата выше этого среднего.

Внутренний запрос возвращает одно число — это и есть скалярный подзапрос.

Многострочный подзапрос

```
SELECT name
FROM employees
WHERE department_id IN (SELECT id FROM departments WHERE region='Европа');
```

Внутренний SELECT возвращает список идентификаторов всех отделов, расположенных в регионе Европа.
Внешний SELECT выбирает сотрудников, чьи department_id присутствуют в этом списке.

Внутренний запрос возвращает несколько строк — это и есть многострочный подзапрос.

Коррелированный подзапрос

```
SELECT e1.name, e1.salary
FROM employees e1
WHERE salary > (
  SELECT AVG(salary)
  FROM employees e2
  WHERE e2.department_id = e1.department_id
);
```

Запрос выполняется для каждой строки **внешнего запроса**.
В этом примере для каждого сотрудника вычисляется средняя зарплата по его отделу, и мы выбираем только тех сотрудников, чья зарплата выше этого значения.

Подзапрос в FROM (временная таблица)

```
SELECT department, avg_salary
FROM (
  SELECT department, AVG(salary) AS
avg_salary
  FROM employees
  GROUP BY department
) AS t
WHERE avg_salary > 50000;
```

Рекомендации по использованию

Задача	Рекомендуемый подзапрос
Вычислить одно агрегатное значение	Скалярный подзапрос
Фильтровать по множеству значений	Многострочный (IN/EXISTS)
Сравнить со средним, максимальным, агрегатом в группе	Коррелированный подзапрос
Подготовить промежуточный набор данных	Подзапрос в FROM

Соединения таблиц (JOIN)

Что это и зачем?

Подзапросы не всегда эффективны

JOIN объединяет таблицы напрямую

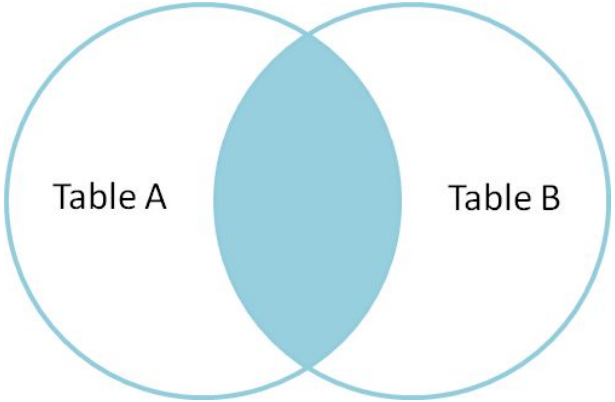
Повышает читаемость и производительность

Используется для аналитики и сложных выборок

Общая структура JOIN-запросов

```
SELECT
    <таблица1.столбец1>,
    <таблица1.столбец2>,
    <таблица2.столбец1>,
    <таблица3.столбец1>,
    ...
FROM <таблица1> AS t1
    [INNER | LEFT | RIGHT | FULL OUTER | CROSS] JOIN <таблица2> AS t2
        ON t1.<ключ_из_таблица1> = t2.<ключ_из_таблица2>
    [INNER | LEFT | RIGHT | FULL OUTER | CROSS] JOIN <таблица3> AS t3
        ON t2.<ключ_из_таблица2> = t3.<ключ_из_таблица3>
    ...
WHERE <условия фильтрации>
GROUP BY <столбцы группировки>
HAVING <условия на агрегаты>
ORDER BY <столбцы сортировки>;
```

INNER JOIN



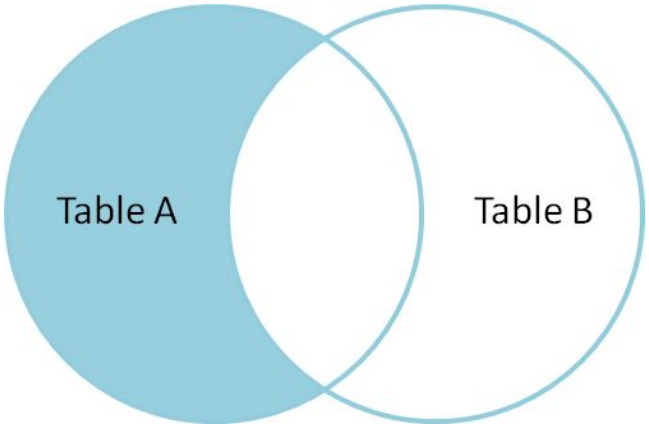
Возвращает **только совпадающие строки** из обеих таблиц

Наиболее часто используемый тип соединения

```
SELECT e.name AS employee_name, d.name AS dep_name
FROM employees e
INNER JOIN departments d ON e.department_id = d.id;
```

INNER JOIN возвращает только совпадающие строки. Если сотрудник не привязан к отделу, он не попадёт в результат.

LEFT JOIN (LEFT OUTER JOIN)



Возвращает все строки из левой таблицы

Если совпадений в правой таблице нет — NULL

```
SELECT e.name AS employee_name, d.name AS dep_name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.id;
```

Все сотрудники будут показаны. Для тех, у кого нет отдела, поле department_name будет NULL.

RIGHT JOIN (RIGHT OUTER JOIN)

Аналогично LEFT JOIN, но для правой таблицы

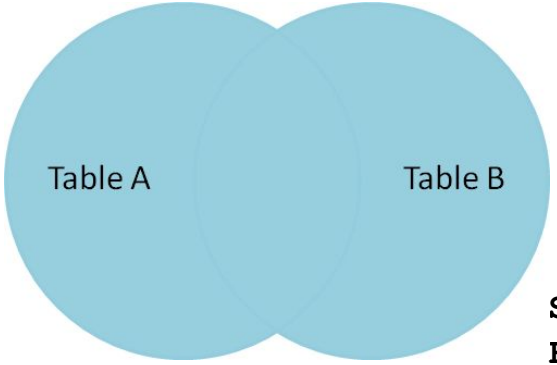
Все строки из правой таблицы + совпадения слева

RIGHT JOIN редко используется, потому что можно просто поменять порядок таблиц и использовать LEFT JOIN

```
SELECT e.name AS employee_name, d.name AS dep_name
FROM employees e
RIGHT JOIN departments d ON e.department_id = d.id;
```

Результат: все отделы, даже пустые.

FULL OUTER JOIN



Объединяет LEFT и RIGHT JOIN

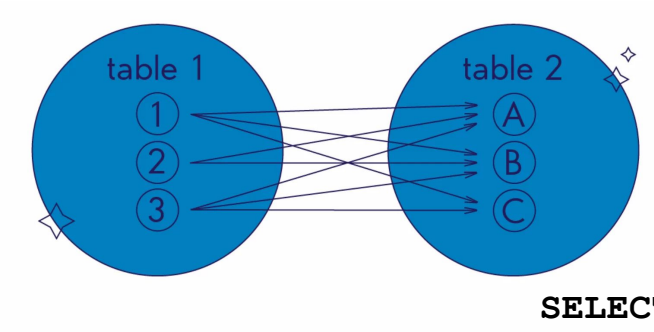
Все строки из обеих таблиц

NULL там, где совпадений нет

```
SELECT e.name AS employee_name, d.name AS dep_name
FROM employees e
FULL OUTER JOIN departments d ON e.department_id = d.id;
```

Полезно для аналитики, когда нужно видеть как сотрудников без отдела, так и отделы без сотрудников.

CROSS JOIN



Декартово произведение двух таблиц

Количество строк = строки в A × строки в B

```
SELECT e.name AS employee_name, p.name AS project_name
FROM employees e
CROSS JOIN projects p;
```

CROSS JOIN создаёт декартово произведение.
Полезно для генерации всех возможных пар «сотрудник — проект», но на больших таблицах может быть очень много строк.

Сравнение JOIN'ов

Тип JOIN	Какие строки возвращает	Когда использовать
INNER JOIN	Только совпадения	Стандартное соединение таблиц
LEFT JOIN	Все левые + совпадения справа	Когда нужны все записи левой таблицы
RIGHT JOIN	Все правые + совпадения слева	Редко, но удобно при правой основной таблице
FULL OUTER JOIN	Все строки обеих таблиц	Для полного отчёта с NULL там, где нет совпадений
CROSS JOIN	Все комбинации строк	Генерация комбинаций, декартово произведение

Операции над множествами в SQL

Зачем нужны операции над множествами

JOIN соединяет строки разных таблиц по условию

UNION, INTERSECT, EXCEPT работают с результатами запросов

Используются для объединения, сравнения или вычитания выборок

UNION — объединение множеств

Объединяет результаты двух SELECT-запросов
Удаляет дубликаты (по умолчанию)
Требует совпадения количества и типов столбцов

```
SELECT e.name AS employee_name, 'Project Participant' AS source
FROM employees e
JOIN employee_projects ep ON e.id = ep.employee_id

UNION
SELECT e.name AS employee_name, 'Sales Performer' AS source
FROM employees e
JOIN sales s ON e.id = s.employee_id;
```

Запрос возвращает общий список всех участников проектов и всех сотрудников, совершавших продажи (с удалением дубликатов).

UNION ALL

Не удаляет дубликаты
Быстрее, чем UNION
Используется, когда важна производительность или нужны повторяющиеся записи

```
SELECT e.name AS employee_name, 'Project Participant' AS source
FROM employees e
JOIN employee_projects ep ON e.id = ep.employee_id

UNION ALL
SELECT e.name AS employee_name, 'Sales Performer' AS source
FROM employees e
JOIN sales s ON e.id = s.employee_id;
```

EXCEPT — разность множеств

Возвращает строки, которые есть в первом запросе, но отсутствуют во втором
Требует совпадения структуры и типов столбцов

```
SELECT e.name FROM employees e
JOIN employee_projects ep ON e.id = ep.employee_id
EXCEPT
SELECT e.name FROM employees e
JOIN sales s ON e.id = s.employee_id;
```

INTERSECT — пересечение множеств

Возвращает только общие строки из двух выборок
Удаляет дубликаты (по умолчанию)
Требует совпадения структуры и типов данных

```
SELECT e.name FROM employees e
JOIN employee_projects ep ON e.id = ep.employee_id
INTERSECT
SELECT e.name FROM employees e
JOIN sales s ON e.id = s.employee_id;
```

Возвращает имена сотрудников которые занимаются и проектами и продажами

Критерий	UNION	UNION ALL	EXCEPT	INTERSECT
Основная идея	Объединяет результаты двух запросов, убирая дубликаты	Объединяет результаты двух запросов, **сохраняя все дубликаты**	Возвращает строки, которые есть в первом запросе, **но отсутствуют во втором**	Возвращает строки, которые есть **в обоих**запросах
Дубликаты	Удаляются автоматически	Сохраняются	Удаляются	Удаляются
Типы данных	Колонки должны иметь одинаковое количество и совместимые типы	То же самое	То же самое	То же самое
Количество столбцов	Должно быть одинаковым в обеих выборках	То же самое	То же самое	То же самое
Порядок столбцов	Должен совпадать	Должен совпадать	Должен совпадать	Должен совпадать
Когда использовать	Когда нужны **уникальные значения** из обеих выборок	Когда нужно **получить все строки**, включая дубликаты (например, статистика по операциям)	Когда нужно **найти различия** между выборками (что есть в одной, но нет в другой)	Когда нужно **найти совпадения**между выборками
Пример	Все сотрудники из отдела продаж **или** маркетинга	Все сотрудники из продаж и маркетинга, включая повторяющихся	Сотрудники, работающие **в продажах, но не в маркетинге**	Сотрудники, работающие **и в продажах, и в маркетинге**